Introducing Grammatically Aware Regular Expression

Tyson Roberts, Rafal Rzepka, Kenji Araki
Language Media Laboratory, Graduate School of Information Science and Technology, Hokkaido University
{nall, kabura, araki}@media.eng.hokudai.ac.jp

Abstract

Regular expression is a common method for matching and extracting information from text. However, it lacks the ability to use linguistic information like part of speech to assist in matching. Addressing this problem, we propose Grammatically Aware Regular Expression (GARE) - a system that replaces character-based alphabets with a part of speech alphabet and a simple filter system. GARE is able overcome many limitations of regular expression. We also introduce MuCha, a GARE implementation written in Python, which we compare to existing systems as well as other existing methods.

Key Words – Language normalization, Information extraction, Regular expression

1. Introduction

The need for powerful tools in modern text processing application is ever increasing. Regular expression (regex), a common shallow parsing method, is unable to perform in places where deeper parsing is required. To address this we introduce grammatically aware regular expression (GARE). GARE is a regular grammar matching system that overcomes difficulties with regex in such situations.

In this paper, we first give an overview of regex and show instances of its shortcomings. In the second section, we will introduce GARE, and show how it can overcome these issues. In the third section, we will introduce MuCha, a system containing an implementation of a GARE system. Next, we show that for a practical NLP task, a GARE system performs well in terms of brevity, expressive power, and extendibility. Finally, we will discuss related systems, and the next step in our research.

2. Regular Expression

Used in a variety of contexts, one solution for matching complex patterns in text is regular expression (regex). A regex is a formal language that describes a particular regular grammar that can thereafter be read by a parser to determine if the input text matches the grammar.

2.1. Overview

Regex is a regular grammar defined over a closed alphabet Σ [1] with $\Sigma \supset \varnothing \bigcup \mathrm{Y} \bigcup \mathrm{T}$. In a natural language context, Y is defined as a set of appropriate glyphs. Y contains items allowed in an input sequence, and defines the items over which a grammar can match.

T is defined as a set of operators taking the form of functions of type $f : \Sigma \rightarrow \mathrm{T}$. Since $\mathrm{T} \subset \Sigma$, it follows that Σ is closed because $\forall f \in \mathrm{T},\ f(a) \in \Sigma$. Regular languages can be denoted by defining the following[1]:

Element	Symbol	Description
empty string	$\varnothing$	The empty string
concatenation	AB	Match A, and then match B at position $p+\lvert A\rvert$.
alternation	$A \mid B$	Matches either A or B at the current position
grouping	(A)	Groups an expression A
literal	$a \in \Sigma$	Any element in Σ
repetition	$A*$	Match A $[0,\infty]$ times over concatenation.

These elements can be combined in many different ways:

$a*b \rightarrow \{b, ab, aab, aaab, aaab, \cdots\}$
$a \mid b \rightarrow \{a, b\}$
$a \mid b* \rightarrow \{a, \varnothing, b, bb, bbb, bbbb, bbbbb, \cdots\}$
$(a \mid b)* \rightarrow \{\varnothing, a, b, abb, bab, babba, aabbab, \cdots\}$

2.2. Drawbacks

Regex is heavily used in shallow parsing situations because of their power and flexibility. One drawback to regex when dealing with natural languages is that affixes and inflections often cause radical surface changes, making creating effective regex unwieldy. Additionally, linguistic information such as part of speech (PoS) and any other tagged data is not usable while matching.

This creates several problems, main of which is that regex is ill suited to linguistic phenomenon that cause surface form to vary. Examples of these problems are as follows, and are discussed are found in the next section:

2.2.1. Searching for verbs in English is difficult. *Chop* is a regular verb, and conjugates normally: *chop* / *chopped* / *chopped*. However, *arise* is irregular, having multiple accepted sets of conjugated forms: *awake* / *awakened* / *awakened* and *awake* / *awoke* / *awoken*. Using regex to find these forms requires alternation on all conjugations.

2.2.2. Grammatical information is not expressed in the surface form of utterances. This is problematic in English, because homographs such as *farm*, *wind*, and *bear* and can express verb or noun forms without context. This can lead to false positives without external intervention.

2.2.3. Tagged information in an utterance such as place or person tags is unavailable when a regex is evaluated, and as such an external system is required to incorporate this information after a completed match has taken place. In this case, regex is of questionable value at all.

2.2.4. Regex systems allow the extraction of match results after execution. The problem with this information is that a match is merely a subsequence containing surface form, devoid of contextual information. These subsequences, then, may lack the required context required to properly interpret them, which can lead to errors.

3. Grammatically Aware Regular Expression

The authors suggest an extension to the regex paradigm: Grammatically Aware Regular Expression (GARE). GARE differs from regex in that it operates over a different alphabet, offers additional domain-specific operators, and allows extraction of items from text from within a grammatical context.

3.1. Overview

The primary difference in a GARE is that the token subset $\Upsilon \subset \Sigma$ on which a GARE operates is not made up of characters, but parts of speech. For instance, one possible alphabet for an English GARE system might be:

$$\Upsilon = \{N, V, ADJ, PREP, CONJ, PUNC, DET\}$$

For instance, for the following utterance:

"My father went to the store yesterday and he departed for a hospital today."

We then use a PoS tagger to make the sequence:

{DET} {N} {V} {PREP} {DET} {N} {ADV} {CONJ} {N} {V} {PREP} {DET} {N} {ADV} {PUNC}

A GARE could then be written to find patterns:

{DET}{N} →
{*My father*, *the store*, *a hospital*}

{V}{ANY}*{V} →
{*went to the store yesterday and he departed*}

({DET}{N}|{N}){V}{PREP} →
{*father went to*, *he departed for*}

Because GARE is constrained to Σ like regex, its ability to match surface forms is restricted. To fix this problem, we propose the concept of filters to GARE.

3.2. Filter System

To overcome this shortcoming of GARE, we propose a filter system. Filters are functions in the form $f : \Sigma \rightarrow \mathrm{B}$, where $B = \{0,1\}$. With this, each element can accept or reject the match based on its filter. Patterns can then incorporate arbitrary criteria. For instance, surface matching:

$f(m) = 1$ if $m_{surface}$ = *store* else 0

{DET}{N: f } →{*store*}

Or to have a requirement on the number of words:

$f(m) = 1$ if $|m_{words}| \leq 4$ else 0

{V}{ANY: f }∗{**N**} →{*departed for a hospital*}

3.3. Addressing Problems

Excessive alternation in **2.2.1**, can be solved by adding a simple filter validate surface forms. The homograph issue in **2.2.2** is solved directly by the nature of GARE. Since PoS is explicitly part of matching, words with different PoS are systemically unambiguous. Filters solve the use of tagged information like place names in **2.2.3** as well. Tags are accessible from filters and can be used as criteria for matching. Finally, extraction issues in **2.2.4** are helped because PoS and context information is kept during matching.

4. MuCha System

Our implementation of a GARE system is integrated into the Japanese language tool suite MuCha, named for the CaboCha project on which the tree parser is based. Though implemented for Japanese, it can be extended to other languages. The system is written in Python and is made up of three modules: the tree module, the normalization module, and the matching module that contains a GARE implementation.

The MuCha system was chosen for the author's experience with the system, the high accuracy of the underlying CaboCha system, and the flexibility of Python.

4.1. Tree Module

The tree module is written on top of the CaboCha dependency structure analyzer. As a first step, MuCha copies CaboCha's shallow chunk / token tree. To achieve a abstraction, each token node is first wrapped in a word node, and then custom combination rules based on Japanese grammar are applied recursively to build up words. Japanese is generally agglutinative, and inflections can change the PoS of the entire word each time an affix is encountered. This can happen repeatedly for highly inflected words.

4.2. Normalization Module

The normalization module generates normalized forms for trees generated by the tree module to provide a stable form for surface forms that differ by affix conjunction.

In Japanese, it is common for inflections to have different conjunctive surface forms according to the root. Additionally, polite verb forms in Japanese regularly use semantically similar conjugations with dramatically different surfaces to plain verb forms. The module solves this problem by creating an extendible regular syntax for representing the Japanese language in a normalized form.

4.3. Matching Module

The matching module's primary role is to allow the creation of matching grammars for Japanese. This module is an implementation of a GARE for the Japanese language. Unlike regex, the grammar is represented directly in program code in the Python language as opposed to a separately coded string.

Specifically, elements of Σ: Y, T, and $\varnothing$ are defined directly as Python objects, and are referred to as tokens, operators, and the ZeroMatch object, respectively.

This approach was chosen for its advantages, including:

- Implementation is simpler than a parsed solution.
- It allows arbitrary Python code to be used in filters
- Readability of grammars is improved
- Grammars can be built dynamically and piecemeal.
- Python object inheritance model allows for user-created tokens, operators, and functions.

4.3.1 MuCha Tokens - Simple

The following tokens match PoS according to their name:

Adverb, Copula, Adjective, NaAdjective
NominalVerb, Noun, Particle, Punctuation, Verb

4.3.2 MuCha Tokens - Auxiliary

Word - Matches any word
Token - Matches any single item, regardless of PoS
Begin - Matches beginning of sequence (zero width)
End - Matches end of sequence (zero width)
ZeroMatch - Matches current position (zero width)
BlankPattern - Special object for recursive grammars

4.3.3 MuCha Operators

Sequence $(e_0 \cdots e_n)$ - Matches elements $e_0 \cdots e_n$ one after another over concatenation.
Choice $(e_0 \cdots e_n, f)$ - Chooses a matching element $e_0 \cdots e_n$ by an arbitrary chooser function f.
Unordered $(e_0 \cdots e_n)$ - Chooses the first matching permutation of $e_0 \cdots e_n$ over concatenation.
All $(e_0 \cdots e_n)$ - Matches e_0 if all elements $e_0 \cdots e_n$ match at the current position.
Any $(e_0 \cdots e_n)$ - Matches the first element in $e_0 \cdots e_n$ that matches at the current position.
Exclusive $(e_0 \cdots e_n)$ - Matches an element in $e_0 \cdots e_n$ if only one element matches.
InverseMatch (e_0) - Matches if e_0 does not match.
Repeat (e_0, i, j) - For $0 \le i \le j$, matches e_0 repeatedly over concatenation $[i, j]$ times.

4.3.4 Filters

Surface (s) - True if surface is s
Normalized (s) - True if normalized form is s
NormalizedRoot (s) - True if normalized root is s
Pronunciation (s) - True if pronunciation is s.
SubMatch (e) - True if element e matches from the beginning of the input match.
SubSearch (e) - True if element e matches at any position in the input match.

For example, matching any surface form of the Japanese verb "eat" would look like:

```
Verb(filter=NormalizedRoot('食べる'))
```

5. Comparison

In this section, we present a comparison of regex and MUCHA algorithms for a text-extraction problem. Algorithms are written in Python. Each program is assumed to have a list of raw sentences in the Japanese language.

The problem is to find an adverb / verb pair located at the end of a sentence. Verbs must be negative, in simple or polite form, and be in any tense. Adverbs are one of the following: ぎりぎり, ギリギリ, すれすれ, スレスレ. The following emotive particles: よ, わ, ね and a single punctuation character are allowed.

5.1. Regular Expression Algorithm

```
pattern = '''
(ぎりぎり|ギリギリ|すれすれ|スレスレ)/ADV\s+
((\S+)
 ((か|さ|ば|ま|ら|わ|が|た|な)?ない|
  (か|さ|ば|ま|ら|わ|が|た|な)?なかった|
  (き|し|び|み|り|い|ぎ|ち|に)?ません|
  (き|し|び|み|り|い|ぎ|ち|に)?ませんでした)/VERB
)|((\S+)/NOUN\s+
  (しない|しなかった|しません|しませんでした)/VERB)
(\S+ (よ|わ|ね)/PART)*\s+
(\S+/PUNC)?$'''

for u in utterances:
  pos = tag_string(u) # EXTERNAL FUNCTION #
  for m in re.findall(pattern, pos):
    print m
```

5.2. GARE Algorithm

```
a_f = Surface('ぎりぎり|ギリギリ|すれすれ|スレスレ')
v_f = Normalized('〜ない|〜ない〜た|〜ます〜ない|〜ない〜ます〜た')
pattern = Adverb(filter=a_f) + Verb(filter=v_f)

for u in utterances:
    for m in mucha.find_all(pattern, u):
        print m
```

5.3. Algorithm Comparison

5.1 uses a long regex to match the adverb-verb pattern directly on a PoS-tagged string. **5.2** uses the MuCha system to describe a sentence pattern and then relies on MuCha to match it. We examine these algorithms in terms of brevity, expressive power, and extendibility.

5.3.1 Brevity. 5.2 is clearly the more concise of the two, expressing the problem in 3 lines, with 3 lines of boilerplate. **5.1** describes the problem in 11 lines and 4 lines of boilerplate (including a call to a PoS tagger).

5.3.2 Expressive Power. 5.1 states it's purpose plainly in terms of characters rather than PoS. **5.2** is arguably more expressive of purpose in this way, describing the PoS and words to match in the output.

5.3.3 Extendibility. Both **5.1** and **5.2** can be changed as easily as changing their input patterns. However, since **5.1** is described in terms of characters, lexical details must be repeated in all future problems, whether or not they are relevant to the problem.

5. Related Works

GARE is a type of tree search, and MuCha can be compared to other systems such as TGrep2. TGrep2 is a search engine for finding structures in tree corpora, and is often used to extract data from the Penn Treebank[3]. Like MuCha, TGrep2 is used to describe patterns that then match against inputs. It also uses many regex-style operators and tokens to describe these patterns.

The fundamental difference between TGrep2 and MuCha is the same as between regex and GARE: alphabet. TGrep2 uses pre-parsed recursive tree structures which describe utterances in terms of phrases. As a result, TGrep2 patterns describe utterances, primarily in terms of nesting of these phrases.

Since TGrep2 uses deeper phrasal parsing, it can be seen as a higher-level tool. On the other hand, MuCha can be used as a tool to find phrase boundaries, implementing a deeper parsing. Finally, TGrep2 does not allow the filters in a way similar to MuCha's filter system.

6. Conclusion

We have described an extension to regex that allows for grammatical awareness for parsing natural languages. This approach has a number of advantages as well as the ability for arbitrary extension. We have shown that this approach is significantly more expressive for natural language, and solves a number of matching cases that simple regex cannot solve on their own.

Additionally, we have completed a reference implementation as a module in the MuCha toolset. This implementation is available online and can be considered a proof-of-concept for the GARE specification.

Lastly, we have shown for a practical example that in terms of brevity, expressive power, and extensibility, our GARE system can be shown to be equal or superior to existing similar solutions.

7. Further Work

For further work, the authors have endeavored to create a system that is easy to write readable grammars, there are situations where automatic pattern detection is useful. We suggest that the MuCha system is a prime candidate for the implementation of such a system due to its ability for manipulation of grammatical forms. Such a system is being considered, and involves a machine learning system using linear programming techniques to construct MuCha-style GARE grammars.

8. References

[1] C. Clarke, G.Cormack (1997). *ACM Transactions on Programming Languages and Systems (TOPLAS)*. pp 413-426

[2] *Official Perl 6 Documentation*, Retrieved from: http://perlcabal.org/syn/

[3] TGrep2 documentation, Retrieved from: http://tedlab.mit.edu/~dr/Tgrep2/